

Тестовое задание: Backend-разработчик

Важно: Ссылку на выполненное тестовое необходимо указать в резюме в блок «О себе»

Срок: 3 дня

Формат: ссылка на репозиторий

Стек: Python (FastAPI) или Java (Spring Boot)

Бизнес-кейс: AI-платформа управления личными подписками и регулярными платежами

Проект: Платформа «Умный реестр подписок» (Backend-ядро и AI-интеграция) для автоматизации учёта, расчёта дат списаний и контроля статуса подписок пользователя.

Проблема: Пользователи ведут учет подписок и регулярных платежей в разрозненных источниках (заметки, календари, почтовые ящики). Единый реестр отсутствует, подписки не имеют уникальных номеров, история изменений не ведется. Расчет дат следующих списаний производится вручную, без учета календарных особенностей (конец месяца, високосный год).

Как процесс работает сейчас (AS IS)

Данные о подписках и регулярных платежах хранятся в разрозненных источниках (заметки, календари, почтовые ящики). Единый реестр обязательств отсутствует, уникальные идентификаторы подписок не фиксируются, история изменений не ведется. Расчет дат следующих списаний производится вручную, без учета календарных особенностей (конец месяца, високосный год). Различение разовых и периодических платежей не производится, статусы подписок обновляются нерегулярно.

Как процесс должен работать (TO BE)

- Пользователь или AI-агент создает обязательство через API, передавая название, сумму, валюту, категорию, периодичность и дату следующего платежа.
- Бэкенд сохраняет данные в PostgreSQL, присваивает уникальный UUID и автоматически проставляет временные метки.
- При создании обязательства с датой в прошлом система сразу присваивает статус expired (а не active), не возвращая ошибку — это позволяет AI-модулю добавлять старые чеки постфактум.
- Если уже существует активное обязательство с таким же названием, бэкенд создает запись, но возвращает предупреждение (warning) вместо ошибки, защищая реестр от сбоев при повторном парсинге писем.
- При оплате рекуррентной подписки дата следующего платежа автоматически сдвигается на период (месяц/квартал/год) с учетом календарных особенностей (31 января + 1 месяц = 28 февраля) без накопления ошибки.
- Перед выдачей списка обязательств бэкенд автоматически применяет логику «ленивого истечения»: просроченные разовые платежи переводятся в expired, а рекуррентные подписки остаются active, так как сервис продолжает действовать.
- При удалении обязательства бэкенд транслирует SSE-событие, чтобы фронтенд обновил интерфейс в реальном времени.

Ключевые сценарии использования (User Stories)

- Как AI-агент, я хочу создавать обязательства через API и получать предупреждение (warning) при обнаружении дубля с таким же названием, чтобы не прерывать процесс обработки писем, но информировать систему о возможном повторе.
- Как система, я хочу автоматически переводить просроченные разовые платежи в статус expired, но оставлять рекуррентные подписки active, чтобы корректно отражать реальное состояние обязательств без ручного вмешательства.

- Как пользователь, я хочу, чтобы при оплате рекуррентной подписки дата следующего платежа автоматически сдвигалась с учётом календарных особенностей (например, с 31 января на 28 февраля), чтобы система корректно работала без накопления ошибок.

Модель данных

Obligations

Поле	Тип	Примечание
id	UUID	генерируется автоматически
title	string	например, «Яндекс.Плюс»
amount	decimal	сумма платежа
currency	string	ISO 4217: RUB, USD, EUR
category	enum	subscription, warranty, bill, insurance
recurrence	enum, nullable	monthly, quarterly, yearly — периодичность повторения; null для разовых обязательств
next_payment_date	date	дата следующего платежа или истечения срока
status	enum	active, cancelled, expired
created_at, updated_at	timestamp	проставляются автоматически

Payments

Отдельная таблица истории оплат. Создаётся через эндпоинт /pay.

Поле	Тип	Примечание
id	UUID	генерируется автоматически
obligation_id	UUID	FK → obligations
amount	decimal	фактически уплаченная сумма
currency	string	валюта платежа
paid_at	timestamp	момент фиксации оплаты

API

POST /obligations

Создаёт новое обязательство. Клиент передаёт все поля кроме id, status, created_at, updated_at.

Правило 1. Если next_payment_date уже в прошлом — обязательство создаётся со статусом expired, а не active. Это не ошибка: AI-модуль может добавить обязательство постфактум, когда распознал старый чек.

Правило 2. Перед созданием проверь, есть ли уже active-обязательство с таким же title (без учёта регистра). Если есть — создай запись, но добавь в ответ поле warning:

```
{
  "obligation": { /* созданная запись */ },
  "warning": "Активное обязательство с таким названием уже существует"
}
```

Это не ошибка: AI-модуль может прислать дубль при повторной обработке одного письма. Клиент решает, что с этим делать.

GET /obligations

Возвращает список обязательств. Принимает опциональные query-параметры category и status, поддерживает их одновременное применение. Результат отсортирован по next_payment_date по возрастанию.

Бизнес-правило (lazy expiry). Перед формированием ответа сервис переводит все записи со статусом active и next_payment_date < today в статус expired. Рекуррентные обязательства (recurrence != null) под это правило не попадают: если пользователь не нажал «Оплачено», дата просто устарела, но подписка не прекратилась. Для них статус остаётся active. Обоснуй этот выбор в README.

GET /obligations/upcoming?days=N

Возвращает обязательства с next_payment_date в диапазоне [today, today + N days]. Параметр days — целое число, по умолчанию 7.

Тело ответа:

```
{
  "obligations": [ /* полный список попавших в окно */ ],
  "totals": {
    "RUB": 1490.00,
    "USD": 9.99
  },
  "renewal_alerts": [
    {
      "id": "...",
      "title": "Netflix",
      "next_payment_date": "2025-06-27",
      "amount": 9.99,
      "currency": "USD"
    }
  ]
}
```

totals — суммы по валютам для всех обязательств в окне. Конвертация на бэкенде не нужна.

renewal_alerts — отдельный список: только подписки (category = subscription) с recurrence != null, чей next_payment_date попадает в окно. Это ключевая фишка продукта — предупредить пользователя, что скоро спишут деньги и ещё можно успеть отменить. Фронтенд и Telegram-бот используют именно этот массив, а не весь список obligations.

POST /obligations/{id}/pay

Фиксирует факт оплаты и обновляет обязательство.

Правило 1. Если статус не active — возвращает 422.

Правило 2. Создаёт запись в таблице payments с текущим amount, currency и paid_at = now().

Правило 3 (рекуррентность). Дальнейшее поведение зависит от поля recurrence:

recurrence	Действие
monthly	next_payment_date += 1 месяц, статус остаётся active
quarterly	next_payment_date += 3 месяца, статус остаётся active
yearly	next_payment_date += 1 год, статус остаётся active
null	статус → cancelled (разовое обязательство закрыто)

Сдвиг считается от текущего next_payment_date, а не от даты оплаты — иначе при просрочке накапливается смещение.

Граничный случай с датами: 31 января + 1 месяц = 28 февраля (или 29 в високосный год), а не ошибка. Используй dateutil.relativedelta или аналог. Опиши поведение в README.

Ответ содержит обновлённое обязательство и созданную запись платежа:

```
{
  "obligation": { /* обновлённая запись */ },
  "payment": { /* созданная запись payments */ }
}
```

PATCH /obligations/{id}/cancel

Переводит обязательство в статус cancelled. Запись остаётся в базе — пользователь видит карточку со статусом cancelled.

Бизнес-правило. Отменить можно только обязательство со статусом active. Попытка отменить expired или уже cancelled → 422 с понятным сообщением.

DELETE /obligations/{id}

Удаляет обязательство и все связанные с ним записи в таблице payments. Возвращает 204 No Content.

Работает для записей в любом статусе — без дополнительных проверок.

После удаления транслирует SSE-событие obligation_deleted:

```
{"type": "obligation_deleted", "id": "..."} 
```

Технические требования

БД: PostgreSQL. Миграции через Alembic или аналог — без ручного CREATE TABLE.

Контейнеризация: Docker Compose. Проект запускается одной командой docker compose up, миграции применяются автоматически при старте.

Документация: OpenAPI. Swagger UI доступен по /docs.

Тесты: pytest. Внешних зависимостей в тестах быть не должно — БД и внешние вызовы мокаются. Обязательно покрыть:

- lazy expiry с учётом исключения для рекуррентных обязательств
- /pay для каждого значения recurrence (включая null)
- граничный случай: оплата 31-го числа с recurrence = monthly
- попытка оплатить или отменить не-active обязательство
- создание дубля и наличие warning в ответе

Что должно быть в README

1. Как запустить проект (docker compose up + применение миграций).
2. Как запустить тесты.
3. Почему lazy expiry не применяется к рекуррентным обязательствам.
4. Как обрабатываются граничные случаи с датами при рекуррентности.
5. Какие компромиссы сделал и что бы сделал иначе при наличии большего времени.
6. Примеры запросов (curl или Postman-коллекция).

2. Мотивация участия в проекте

1. Что тебя привлекает в этом проекте?
2. Как ты видишь свою роль в команде, которая строит AI-продукт?
3. Сколько часов в неделю готов уделять и на какой срок?